Assignment No. 3

Title of the Experiment:
Creating an Application in SalesForce.com using Apex Programming Language

Instructor's Name: Prof. S.N.Girme

Student's Name: Sukanya Gupta

Student's Roll no: 31416

Objectives:

- The objective of this experiment was to create a Scientific Calculator application on the Salesforce.com platform using Apex programming language. The application uses an Apex Controller class to implement arithmetic and scientific operations, and a Visualforce Page as the user interface — demonstrating the use of Salesforce's cloud-based PaaS (Platform as a Service) for building custom web applications.

Software & Resources Used:

- Platform: Salesforce.com (Developer Edition Org)
- Org URL: orgfarm-d69ff1e6a2-dev-ed.develop.my.salesforce.com
- IDE/Editor: Salesforce Developer Console (in-browser)
- Backend Language: Apex (API Version 66)
- Frontend: Visualforce Pages (.vfp) with apex:page, apex:form, apex:commandButton components
- Application Name: Apex Scientific Calculator
- Browser: Google Chrome

Theory / Background:

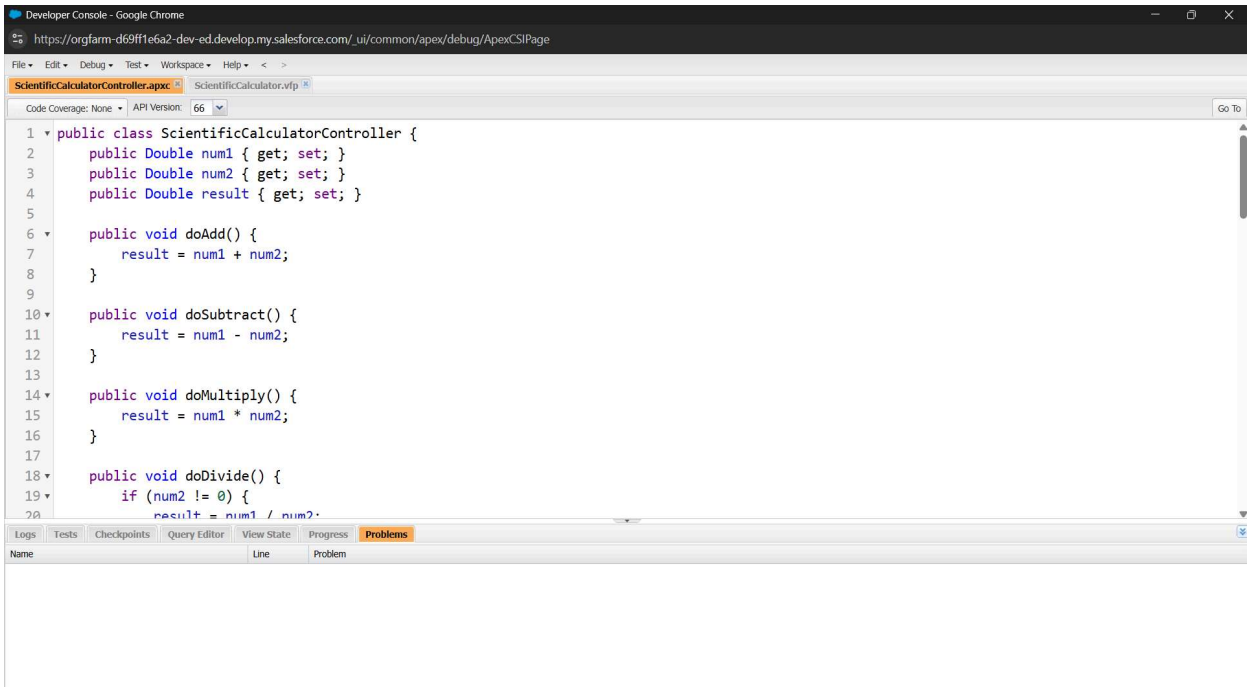
Salesforce is a cloud-based CRM (Customer Relationship Management) platform that also provides a powerful PaaS environment for building custom applications. Apex is Salesforce's proprietary, strongly-typed, object-oriented programming language that runs on the Salesforce servers (similar to Java). It is used to write business logic, controllers, and triggers.

Visualforce is Salesforce's markup language for building user interfaces. A Visualforce Page (.vfp) uses HTML-like tags prefixed with apex: to bind directly to an Apex controller. The controller exposes properties and action methods that the page can read and invoke.

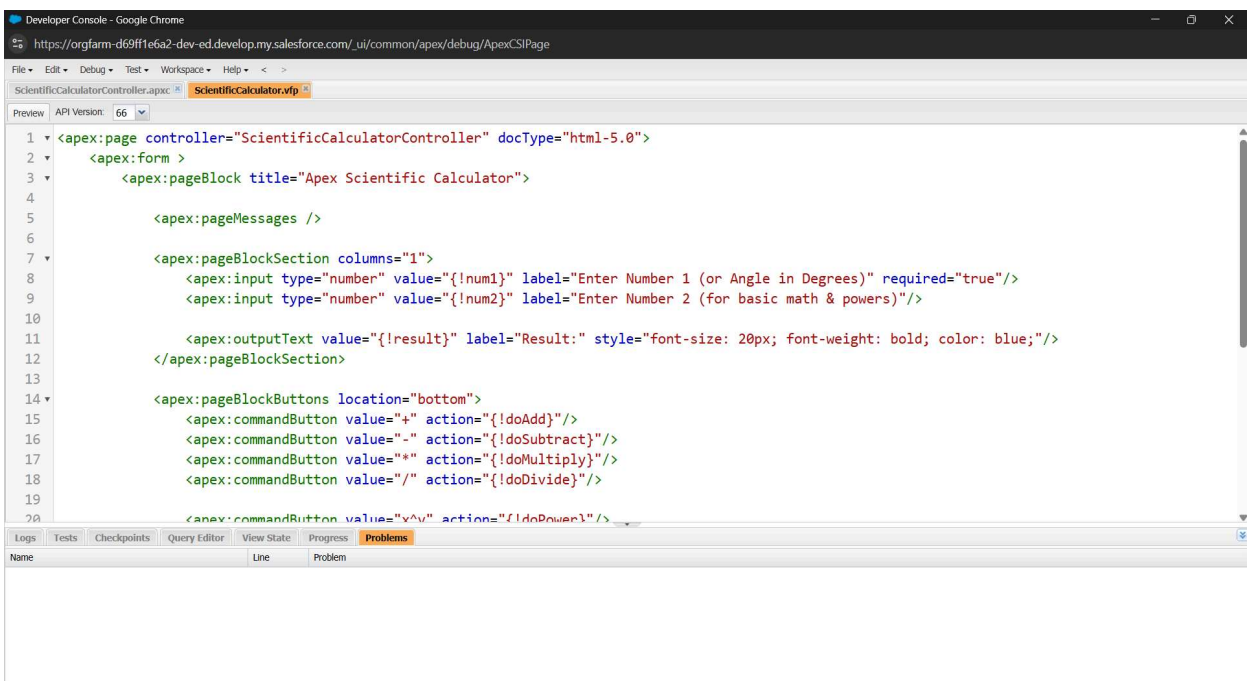
In this experiment, a Scientific Calculator was built with two components: (1) ScientificCalculatorController.apxc the Apex class that holds the logic for arithmetic operations (add, subtract, multiply, divide), power, negation, modulo, square root, sine, cosine, tangent, and natural logarithm; and (2) ScientificCalculator.vfp the Visualforce page that renders the input fields, result display, and operation buttons.

Steps / Procedure with Screenshots:

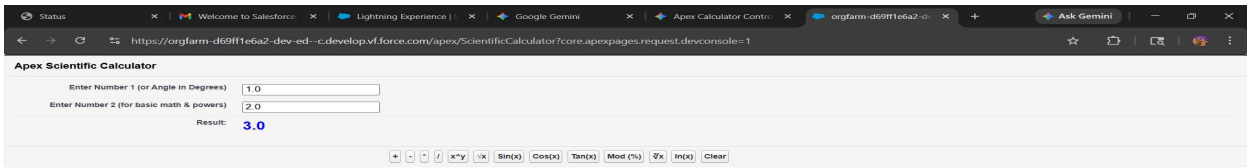
Step 1: We opened the Salesforce Developer Console and created a new Apex Class named 'ScientificCalculatorController'. The class defines three public Double properties num1, num2, and result with getter/setter methods. It implements action methods for each operation: doAdd() (addition), doSubtract() (subtraction), doMultiply() (multiplication), and doDivide() (division with a check for division by zero), along with scientific functions for power, negation, modulus, square root, sin, cos, tan, and ln.



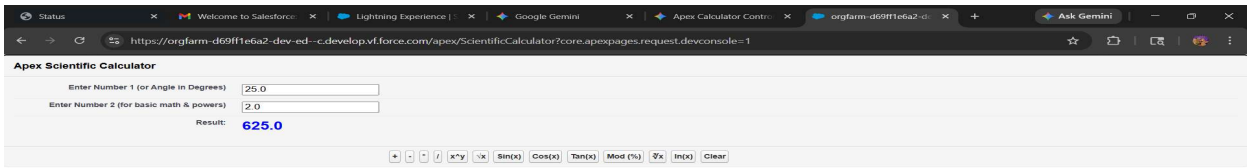
Step 2: We created the Visualforce Page named 'ScientificCalculator.vfp' linked to the controller using controller="ScientificCalculatorController". The page uses apex:pageBlock with a title 'Apex Scientific Calculator', apex:input fields bound to {!num1} and {!num2}, and an apex:outputText to display {!result} in blue bold style (font-size: 20px). The apex:pageBlockButtons section at the bottom renders all operation buttons (+, -, *, /, x^y, √x, Sin, Cos, Tan, Mod, ln, Clear) as apex:commandButton elements, each wired to their respective controller action.



Step 3: We tested the calculator by entering Number 1 = 1.0 and Number 2 = 2.0 and clicking the '+' button. The Apex doAdd() method executed on the Salesforce server and returned the result 3.0, which was displayed in blue on the page. This confirmed that the basic arithmetic addition operation was working correctly.



Step 4: We tested the power function by entering Number 1 = 25.0 and Number 2 = 2.0 and clicking the 'x^y' button. The Apex doPower() method computed 25.0 raised to the power of 2.0 and returned 625.0. The result was displayed in bold blue, confirming that the scientific operations were functioning correctly on the Salesforce platform.



Conclusion

In this experiment, we successfully created a Scientific Calculator application on the Salesforce.com platform using Apex programming language and Visualforce pages. We created an Apex controller class implementing arithmetic operations (+, -, *, /) as well as scientific functions (power, square root, sine, cosine, tangent, modulo, and natural logarithm). The Visualforce page served as the UI, binding input fields and buttons directly to controller methods. The application was tested with addition (1.0 + 2.0 = 3.0) and power operations (25.0 ^ 2.0 = 625.0), both returning correct results. This demonstrated how Salesforce's PaaS environment enables rapid development and deployment of custom cloud-based applications without managing any infrastructure.